HOME WORK SET 6

Data Structures & Algorithms

Programming Language: C

Roll Number: 2025115140

17 Detailed C Programs with Executed Output Screenshots

Submitted for GitHub / Lab Record

1. Simulate Undo and Redo in a word-processing package using a doubly linked list.

Program: 01_dll_undo_redo.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TEXT 100

typedef struct Node {
    char text[MAX_TEXT];
    struct Node *prev;
    struct Node *next;
} Node;

Node *current = NULL;

void saveState(const char *text)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    strcpy(newNode->text, text);
    newNode->prev = current;
    newNode->next = NULL;

    if (current != NULL)
        current->next = newNode;

    current = newNode;
}

void undo(void)
{
    if (current != NULL && current->prev != NULL) {
        current = current->prev;
        printf("Undo successful.\n");
    } else {
        printf("Nothing to undo.\n");
    }
}

void redo(void)
{
    if (current != NULL && current->next != NULL) {
        current = current->next;
        printf("Redo successful.\n");
    } else {
        printf("Nothing to redo.\n");
    }
}

void displayCurrent(void)
{
    if (current != NULL)
        printf("Current text: %s\n", current->text);
    else
        printf("No text available.\n");
}

int main(void)
{
    int choice;
    char text[MAX_TEXT];

    do {
        printf("\n--- UNDO / REDO ---\n");
        printf("1. Save new state\n");
        printf("2. Undo\n");
        printf("3. Redo\n");
        printf("4. Display current text\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();

        switch (choice) {
        case 1:
            printf("Enter text: ");
            fgets(text, MAX_TEXT, stdin);
            text[strcspn(text, "\n")] = '\0';
```

```
            saveState(text);
            break;

        case 2:
            undo();
            break;

        case 3:
            redo();
            break;

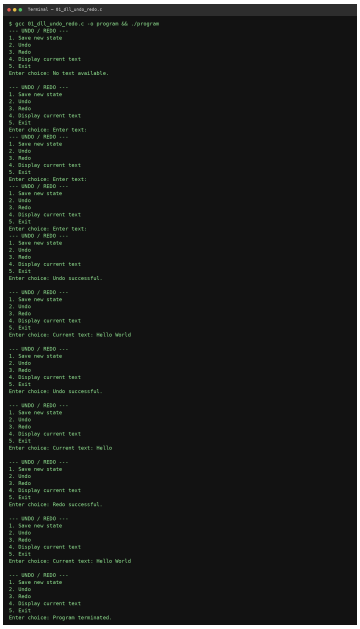
        case 4:
            displayCurrent();
            break;

        case 5:
            printf("Program terminated.\n");
            break;

        default:
            printf("Invalid choice.\n");
        }
    } while (choice != 5);

    return 0;
}
```

Executed Output



2. Given two sorted lists L1 and L2, compute L1 ∩ L2 and L1 ∪ L2 using basic linked-list operations.

Program: 02_union_intersection.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void insertEnd(Node **head, int value)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    Node *temp;

    newNode->data = value;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

void display(Node *head)
{
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

Node *intersection(Node *a, Node *b)
{
    Node *result = NULL;

    while (a != NULL && b != NULL) {
        if (a->data == b->data) {
            insertEnd(&result, a->data);
            a = a->next;
            b = b->next;
        } else if (a->data < b->data) {
            a = a->next;
        } else {
            b = b->next;
        }
    }

    return result;
}

Node *unionList(Node *a, Node *b)
{
    Node *result = NULL;
    int last = 0;
    int hasLast = 0;

    while (a != NULL || b != NULL) {
        int value;

        if (b == NULL || (a != NULL && a->data < b->data)) {
            value = a->data;
            a = a->next;
        } else if (a == NULL || b->data < a->data) {
            value = b->data;
            b = b->next;
        } else {
            value = a->data;
            a = a->next;
            b = b->next;
        }

        if (!hasLast || value != last) {
            insertEnd(&result, value);
            last = value;
            hasLast = 1;
        }
```

```
    }

    return result;
}

int main(void)
{
    Node *L1 = NULL, *L2 = NULL;
    Node *I, *U;
    int n, m, value, i;

    printf("Enter number of elements in L1: ");
    scanf("%d", &n);
    printf("Enter sorted L1: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(&L1, value);
    }

    printf("Enter number of elements in L2: ");
    scanf("%d", &m);
    printf("Enter sorted L2: ");
    for (i = 0; i < m; i++) {
        scanf("%d", &value);
        insertEnd(&L2, value);
    }

    I = intersection(L1, L2);
    U = unionList(L1, L2);

    printf("L1: ");
    display(L1);
    printf("L2: ");
    display(L2);
    printf("Intersection: ");
    display(I);
    printf("Union: ");
    display(U);

    return 0;
}
```

Executed Output



3. Swap two elements of a doubly linked list by changing only pointers (not data).

Program: 03_swap_dll_nodes.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

void insertEnd(Node **head, int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    Node *t;

    p->data = value;
    p->prev = NULL;
    p->next = NULL;

    if (*head == NULL) {
        *head = p;
        return;
    }

    t = *head;
    while (t->next != NULL)
        t = t->next;

    t->next = p;
    p->prev = t;
}

Node *findNode(Node *head, int value)
{
    while (head != NULL && head->data != value)
        head = head->next;

    return head;
}

void swapNodes(Node **head, Node *a, Node *b)
{
    Node *ap, *an, *bp, *bn;

    if (a == NULL || b == NULL || a == b)
        return;

    /* Make a appear before b. */
    if (b->prev == a) {
        Node *temp = a;
        a = b;
        b = temp;
    }

    ap = a->prev;
    an = a->next;
    bp = b->prev;
    bn = b->next;

    if (an == b) {
        if (ap != NULL)
            ap->next = b;
        else
            *head = b;

        if (bn != NULL)
            bn->prev = a;

        b->prev = ap;
        b->next = a;
        a->prev = b;
        a->next = bn;
    } else {
        if (ap != NULL)
            ap->next = b;
        else
            *head = b;

        if (an != NULL)
            an->prev = b;

        if (bp != NULL)
            bp->next = a;
```

```
        else
            *head = a;

        if (bn != NULL)
            bn->prev = a;

        a->prev = bp;
        a->next = bn;
        b->prev = ap;
        b->next = an;
    }
}

void display(Node *head)
{
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main(void)
{
    Node *head = NULL;
    Node *a, *b;
    int n, value, x, y, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(&head, value);
    }

    printf("Enter two elements to swap: ");
    scanf("%d%d", &x, &y);

    a = findNode(head, x);
    b = findNode(head, y);

    printf("Before swap: ");
    display(head);

    swapNodes(&head, a, b);

    printf("After swap: ");
    display(head);

    return 0;
}
```

Executed Output

```
Terminal — 03_swap_dll_nodes.c

$ gcc 03_swap_dll_nodes.c -o program && ./program
Enter number of nodes: Enter elements: Enter two elements to swap: Before
swap: 10 20 30 40 50
After swap: 10 40 30 20 50
```

4. Write a self-adjusting doubly linked list. When an element is accessed, move it to the front without changing the order of the other elements.

Program: 04_self_adjusting_dll.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

void insertEnd(Node **head, int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    Node *t;

    p->data = value;
    p->prev = NULL;
    p->next = NULL;

    if (*head == NULL) {
        *head = p;
        return;
    }

    t = *head;
    while (t->next != NULL)
        t = t->next;

    t->next = p;
    p->prev = t;
}

void accessElement(Node **head, int key)
{
    Node *p = *head;

    while (p != NULL && p->data != key)
        p = p->next;

    if (p == NULL) {
        printf("Element not found.\n");
        return;
    }

    if (p == *head) {
        printf("Element is already at the front.\n");
        return;
    }

    p->prev->next = p->next;

    if (p->next != NULL)
        p->next->prev = p->prev;

    p->prev = NULL;
    p->next = *head;
    (*head)->prev = p;
    *head = p;
}

void display(Node *head)
{
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main(void)
{
    Node *head = NULL;
    int n, value, key, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(&head, value);
    }
```

```
    printf("Enter accessed element: ");
    scanf("%d", &key);

    printf("Before access: ");
    display(head);

    accessElement(&head, key);

    printf("After access: ");
    display(head);

    return 0;
}
```

Executed Output



5. Write a C program to reverse a doubly linked list.

Program: 05_reverse_dll.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

void insertEnd(Node **head, int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    Node *t;

    p->data = value;
    p->prev = NULL;
    p->next = NULL;

    if (*head == NULL) {
        *head = p;
        return;
    }

    t = *head;
    while (t->next != NULL)
        t = t->next;

    t->next = p;
    p->prev = t;
}

void reverse(Node **head)
{
    Node *current = *head;
    Node *temp;

    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;

        *head = current;
        current = current->prev;
    }
}

void display(Node *head)
{
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main(void)
{
    Node *head = NULL;
    int n, value, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(&head, value);
    }

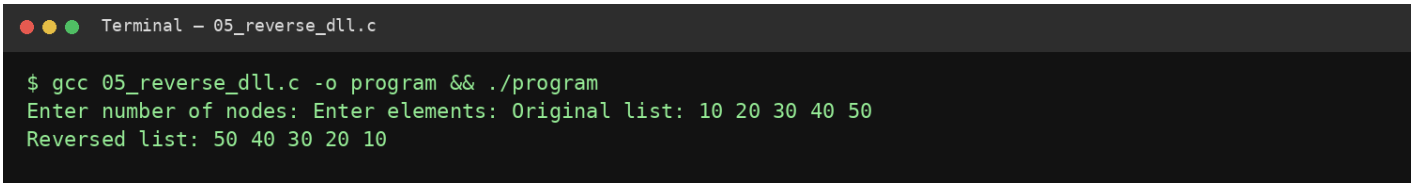
    printf("Original list: ");
    display(head);

    reverse(&head);

    printf("Reversed list: ");
    display(head);

    return 0;
}
```

Executed Output


Terminal — 05_reverse_dll.c
$ gcc 05_reverse_dll.c -o program && ./program
Enter number of nodes: Enter elements: Original list: 10 20 30 40 50
Reversed list: 50 40 30 20 10

6. Write a C program to delete the middle element of a doubly linked list.

Program: 06_delete_middle_dll.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

void insertEnd(Node **head, int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    Node *t;

    p->data = value;
    p->prev = NULL;
    p->next = NULL;

    if (*head == NULL) {
        *head = p;
        return;
    }

    t = *head;
    while (t->next != NULL)
        t = t->next;

    t->next = p;
    p->prev = t;
}

void deleteMiddle(Node **head)
{
    Node *slow = *head;
    Node *fast = *head;

    if (*head == NULL)
        return;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    if (slow->prev != NULL)
        slow->prev->next = slow->next;
    else
        *head = slow->next;

    if (slow->next != NULL)
        slow->next->prev = slow->prev;

    free(slow);
}

void display(Node *head)
{
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main(void)
{
    Node *head = NULL;
    int n, value, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(&head, value);
    }

    printf("Before deletion: ");
    display(head);

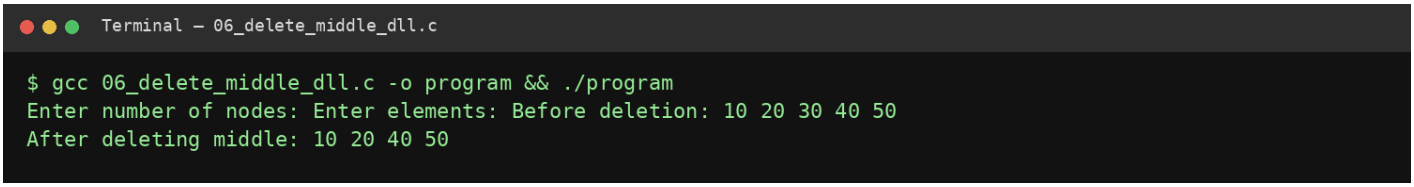
    deleteMiddle(&head);

    printf("After deleting middle: ");
```

```
    display(head);

    return 0;
}
```

Executed Output



7. Remove duplicate elements from (i) a sorted DLL and (ii) an unsorted DLL.

Program: 07_remove_duplicates_dll.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

void insertEnd(Node **head, int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    Node *t;

    p->data = value;
    p->prev = NULL;
    p->next = NULL;

    if (*head == NULL) {
        *head = p;
        return;
    }

    t = *head;
    while (t->next != NULL)
        t = t->next;

    t->next = p;
    p->prev = t;
}

void removeSortedDuplicates(Node *head)
{
    Node *current = head;
    Node *duplicate;

    while (current != NULL && current->next != NULL) {
        if (current->data == current->next->data) {
            duplicate = current->next;
            current->next = duplicate->next;

            if (duplicate->next != NULL)
                duplicate->next->prev = current;

            free(duplicate);
        } else {
            current = current->next;
        }
    }
}

void removeUnsortedDuplicates(Node *head)
{
    Node *current, *runner, *duplicate;

    for (current = head; current != NULL; current = current->next) {
        runner = current->next;

        while (runner != NULL) {
            duplicate = runner;
            runner = runner->next;

            if (duplicate->data == current->data) {
                if (duplicate->prev != NULL)
                    duplicate->prev->next = duplicate->next;

                if (duplicate->next != NULL)
                    duplicate->next->prev = duplicate->prev;

                free(duplicate);
            }
        }
    }
}

void display(Node *head)
{
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
```

```
}

int main(void)
{
    Node *head = NULL;
    int type, n, value, i;

    printf("1. Sorted list\n");
    printf("2. Unsorted list\n");
    printf("Enter choice: ");
    scanf("%d", &type);

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(&head, value);
    }

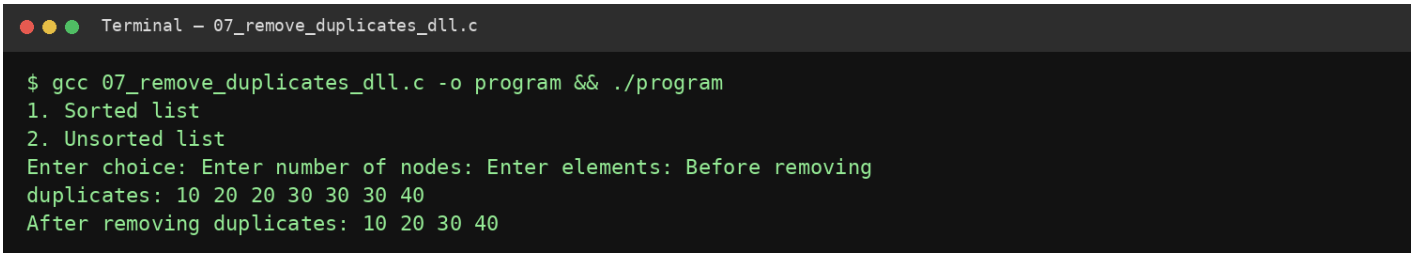
    printf("Before removing duplicates: ");
    display(head);

    if (type == 1)
        removeSortedDuplicates(head);
    else
        removeUnsortedDuplicates(head);

    printf("After removing duplicates: ");
    display(head);

    return 0;
}
```

Executed Output



8. Calculate the frequency count of a given integer in a circular doubly linked list.

Program: 08_frequency_circular_dll.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

void insertEnd(Node **head, int value)
{
    Node *p = (Node *)malloc(sizeof(Node));

    p->data = value;

    if (*head == NULL) {
        p->next = p;
        p->prev = p;
        *head = p;
        return;
    }

    Node *tail = (*head)->prev;

    p->next = *head;
    p->prev = tail;
    tail->next = p;
    (*head)->prev = p;
}

int frequency(Node *head, int key)
{
    Node *current;
    int count = 0;

    if (head == NULL)
        return 0;

    current = head;

    do {
        if (current->data == key)
            count++;

        current = current->next;
    } while (current != head);

    return count;
}

void display(Node *head)
{
    Node *current;

    if (head == NULL)
        return;

    current = head;

    do {
        printf("%d ", current->data);
        current = current->next;
    } while (current != head);

    printf("\n");
}

int main(void)
{
    Node *head = NULL;
    int n, value, key, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(&head, value);
    }

    printf("Circular DLL: ");
```

```
    display(head);

    printf("Enter element to find frequency: ");
    scanf("%d", &key);

    printf("Frequency of %d = %d\n", key, frequency(head, key));

    return 0;
}
```

Executed Output

```
Terminal — 08_frequency_circular_dll.c

$ gcc 08_frequency_circular_dll.c -o program && ./program
Enter number of nodes: Enter elements: Circular DLL: 10 20 10 30 10 40 20
Enter element to find frequency: Frequency of 10 = 3
```

9. Search for a particular element in a binary tree.

Program: 09_search_binary_tree.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *createNode(int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = value;
    p->left = NULL;
    p->right = NULL;
    return p;
}

int search(Node *root, int key)
{
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    return search(root->left, key) ||
           search(root->right, key);
}

int main(void)
{
    Node *root;
    int key;

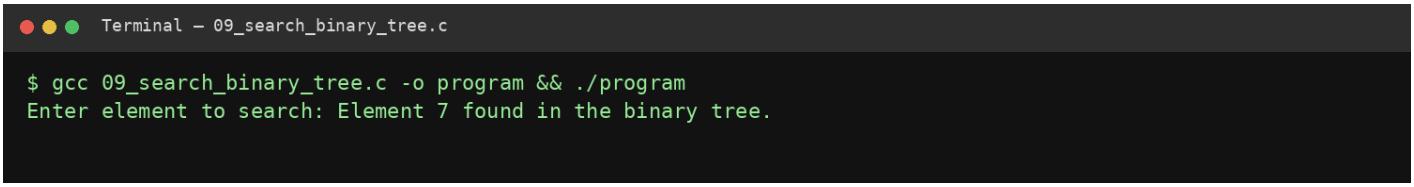
    root = createNode(10);
    root->left = createNode(5);
    root->right = createNode(15);
    root->left->left = createNode(3);
    root->left->right = createNode(7);
    root->right->left = createNode(12);
    root->right->right = createNode(20);

    printf("Enter element to search: ");
    scanf("%d", &key);

    if (search(root, key))
        printf("Element %d found in the binary tree.\n", key);
    else
        printf("Element %d not found in the binary tree.\n", key);

    return 0;
}
```

Executed Output



10. Calculate the height/depth of a binary tree and print it.

Program: 10_height_binary_tree.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *createNode(int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = value;
    p->left = NULL;
    p->right = NULL;
    return p;
}

int height(Node *root)
{
    int leftHeight, rightHeight;

    if (root == NULL)
        return 0;

    leftHeight = height(root->left);
    rightHeight = height(root->right);

    if (leftHeight > rightHeight)
        return leftHeight + 1;

    return rightHeight + 1;
}

int main(void)
{
    Node *root = createNode(10);

    root->left = createNode(5);
    root->right = createNode(15);
    root->left->left = createNode(3);
    root->left->right = createNode(7);
    root->right->right = createNode(20);

    printf("Height of binary tree = %d\n", height(root));

    return 0;
}
```

Executed Output

```
Terminal — 10_height_binary_tree.c

$ gcc 10_height_binary_tree.c -o program && ./program
Height of binary tree = 3
```

11. Convert a binary tree to a doubly linked list while preserving inorder order.

Program: 11_binary_tree_to_dll.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *head = NULL;
Node *previous = NULL;

Node *createNode(int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = value;
    p->left = NULL;
    p->right = NULL;
    return p;
}

void convertToDLL(Node *root)
{
    if (root == NULL)
        return;

    convertToDLL(root->left);

    if (previous == NULL)
        head = root;
    else {
        root->left = previous;
        previous->right = root;
    }

    previous = root;

    convertToDLL(root->right);
}

void displayDLL(Node *head)
{
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->right;
    }
    printf("\n");
}

int main(void)
{
    Node *root = createNode(10);

    root->left = createNode(5);
    root->right = createNode(15);
    root->left->left = createNode(3);
    root->left->right = createNode(7);

    convertToDLL(root);

    printf("DLL in inorder: ");
    displayDLL(head);

    return 0;
}
```

Executed Output

```
Terminal — 11_binary_tree_to_dll.c

$ gcc 11_binary_tree_to_dll.c -o program && ./program
DLL in inorder: 3 5 7 10 15
```

12. Given a binary tree, connect nodes at the same level using a next-right pointer.

Program: 12_connect_same_level.c

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
    struct Node *nextRight;
} Node;

Node *createNode(int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = value;
    p->left = NULL;
    p->right = NULL;
    p->nextRight = NULL;
    return p;
}

void connectSameLevel(Node *root)
{
    Node *queue[MAX];
    int front = 0, rear = 0;
    int levelCount, i;
    Node *current, *previous;

    if (root == NULL)
        return;

    queue[rear++] = root;

    while (front < rear) {
        levelCount = rear - front;
        previous = NULL;

        for (i = 0; i < levelCount; i++) {
            current = queue[front++];

            if (previous != NULL)
                previous->nextRight = current;

            previous = current;

            if (current->left != NULL)
                queue[rear++] = current->left;

            if (current->right != NULL)
                queue[rear++] = current->right;
        }

        previous->nextRight = NULL;
    }
}

void printLevels(Node *root)
{
    Node *queue[MAX];
    int front = 0, rear = 0;
    int levelCount, i;
    Node *current;

    if (root == NULL)
        return;

    queue[rear++] = root;

    while (front < rear) {
        levelCount = rear - front;

        for (i = 0; i < levelCount; i++) {
            current = queue[front++];
            printf("%d ", current->data);

            if (current->left != NULL)
                queue[rear++] = current->left;

            if (current->right != NULL)
                queue[rear++] = current->right;
        }
```

```
        printf("\n");
    }
}

int main(void)
{
    Node *root = createNode(1);

    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);
    root->right->right = createNode(6);

    connectSameLevel(root);

    printf("Levels after connecting next-right pointers:\n");
    printLevels(root);

    printf("\nLevel 2 links: %d -> %d -> NULL\n",
           root->left->data,
           root->left->nextRight->data);

    printf("Level 3 links: %d -> %d -> %d -> NULL\n",
           root->left->left->data,
           root->left->left->nextRight->data,
           root->left->left->nextRight->nextRight->data);

    return 0;
}
```

Executed Output

```
Terminal — 12_connect_same_level.c

$ gcc 12_connect_same_level.c -o program && ./program
Levels after connecting next-right pointers:
1
2 3
4 5 6

Level 2 links: 2 -> 3 -> NULL
Level 3 links: 4 -> 5 -> 6 -> NULL
```

13. Implement a priority queue using a binary search tree with insertion and deletion operations.

Program: 13_priority_queue_bst.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int priority;
    struct Node *left;
    struct Node *right;
} Node;

Node *createNode(int priority)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->priority = priority;
    p->left = NULL;
    p->right = NULL;
    return p;
}

Node *insert(Node *root, int priority)
{
    if (root == NULL)
        return createNode(priority);

    if (priority < root->priority)
        root->left = insert(root->left, priority);
    else
        root->right = insert(root->right, priority);

    return root;
}

Node *findMinimum(Node *root)
{
    while (root->left != NULL)
        root = root->left;
    return root;
}

Node *deletePriority(Node *root, int priority)
{
    Node *temp;

    if (root == NULL)
        return NULL;

    if (priority < root->priority)
        root->left = deletePriority(root->left, priority);
    else if (priority > root->priority)
        root->right = deletePriority(root->right, priority);
    else {
        if (root->left == NULL) {
            temp = root->right;
            free(root);
            return temp;
        }

        if (root->right == NULL) {
            temp = root->left;
            free(root);
            return temp;
        }

        temp = findMinimum(root->right);
        root->priority = temp->priority;
        root->right =
            deletePriority(root->right, temp->priority);
    }

    return root;
}

int getHighestPriority(Node *root)
{
    while (root->right != NULL)
        root = root->right;

    return root->priority;
}

void inorder(Node *root)
{
    if (root != NULL) {
```

```
        inorder(root->left);
        printf("%d ", root->priority);
        inorder(root->right);
    }
}

int main(void)
{
    Node *root = NULL;
    int n, value, deleteValue, i;

    printf("Enter number of priorities: ");
    scanf("%d", &n);

    printf("Enter priorities: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("BST priorities: ");
    inorder(root);

    printf("\nHighest priority = %d\n",
           getHighestPriority(root));

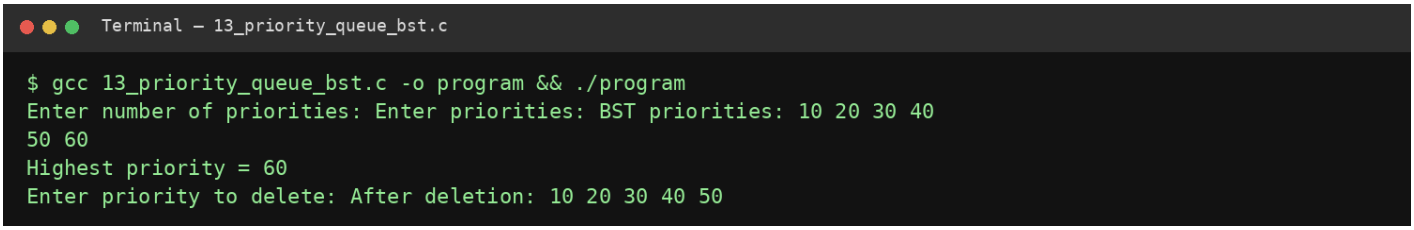
    printf("Enter priority to delete: ");
    scanf("%d", &deleteValue);

    root = deletePriority(root, deleteValue);

    printf("After deletion: ");
    inorder(root);
    printf("\n");

    return 0;
}
```

Executed Output



14. In a given binary search tree, delete a node having zero, one or two children.

Program: 14_bst_delete.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *createNode(int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = value;
    p->left = NULL;
    p->right = NULL;
    return p;
}

Node *insert(Node *root, int value)
{
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

    return root;
}

Node *minimumNode(Node *root)
{
    Node *current = root;

    while (current->left != NULL)
        current = current->left;

    return current;
}

Node *deleteNode(Node *root, int key)
{
    Node *temp;

    if (root == NULL)
        return NULL;

    if (key < root->data)
        root->left = deleteNode(root->left, key);

    else if (key > root->data)
        root->right = deleteNode(root->right, key);

    else {
        /* Case 1 and Case 2: zero or one child */
        if (root->left == NULL) {
            temp = root->right;
            free(root);
            return temp;
        }

        if (root->right == NULL) {
            temp = root->left;
            free(root);
            return temp;
        }

        /* Case 3: two children */
        temp = minimumNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}

void inorder(Node *root)
{
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
```

```
        inorder(root->right);
    }
}

int main(void)
{
    Node *root = NULL;
    int n, value, key, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter BST elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("Inorder before deletion: ");
    inorder(root);

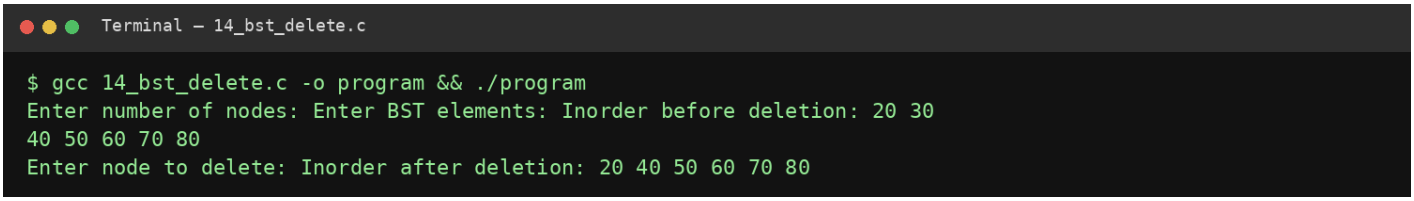
    printf("\nEnter node to delete: ");
    scanf("%d", &key);

    root = deleteNode(root, key);

    printf("Inorder after deletion: ");
    inorder(root);
    printf("\n");

    return 0;
}
```

Executed Output



15. Convert a given postfix expression to an expression tree.

Program: 15_postfix_expression_tree.c

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100

typedef struct Node {
    char data;
    struct Node *left;
    struct Node *right;
} Node;

Node *stack[MAX];
int top = -1;

Node *createNode(char value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = value;
    p->left = NULL;
    p->right = NULL;
    return p;
}

void push(Node *node)
{
    stack[++top] = node;
}

Node *pop(void)
{
    return stack[top--];
}

Node *buildExpressionTree(char postfix[])
{
    int i;
    Node *root;

    for (i = 0; postfix[i] != '\0'; i++) {
        if (isalnum((unsigned char)postfix[i])) {
            push(createNode(postfix[i]));
        } else {
            root = createNode(postfix[i]);

            root->right = pop();
            root->left = pop();

            push(root);
        }
    }

    return pop();
}

void inorder(Node *root)
{
    if (root == NULL)
        return;

    if (root->left != NULL)
        printf("(");

    inorder(root->left);
    printf("%c", root->data);
    inorder(root->right);

    if (root->right != NULL)
        printf(")");
}

int main(void)
{
    char postfix[MAX];
    Node *root;

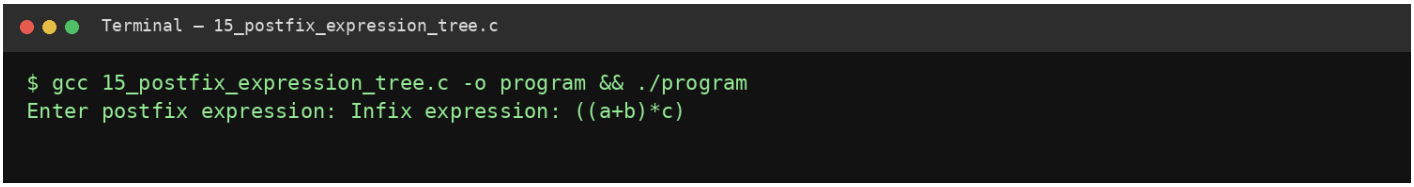
    printf("Enter postfix expression: ");
    scanf("%99s", postfix);

    root = buildExpressionTree(postfix);

    printf("Infix expression: ");
    inorder(root);
    printf("\n");
```

```
    return 0;
}
```

Executed Output



16. Construct a binary search tree and display its inorder, preorder and postorder traversals.

Program: 16_bst_traversals.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *createNode(int value)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = value;
    p->left = NULL;
    p->right = NULL;
    return p;
}

Node *insert(Node *root, int value)
{
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

    return root;
}

void inorder(Node *root)
{
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(Node *root)
{
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(Node *root)
{
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main(void)
{
    Node *root = NULL;
    int n, value, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("Inorder: ");
    inorder(root);

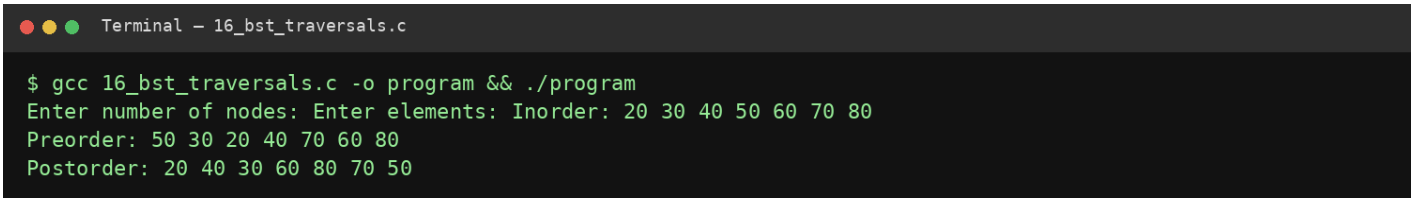
    printf("\nPreorder: ");
    preorder(root);

    printf("\nPostorder: ");
    postorder(root);

    printf("\n");
```

```
    return 0;
}
```

Executed Output



17. Implement heap sort for a given set of numbers.

Program: 17_heap_sort.c

```c
#include <stdio.h>
#include <stdlib.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int array[], int n, int index)
{
    int largest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < n && array[left] > array[largest])
        largest = left;

    if (right < n && array[right] > array[largest])
        largest = right;

    if (largest != index) {
        swap(&array[index], &array[largest]);
        heapify(array, n, largest);
    }
}

void heapSort(int array[], int n)
{
    int i;

    /* Build max heap */
    for (i = n / 2 - 1; i >= 0; i--)
        heapify(array, n, i);

    /* Extract maximum repeatedly */
    for (i = n - 1; i > 0; i--) {
        swap(&array[0], &array[i]);
        heapify(array, i, 0);
    }
}

int main(void)
{
    int n, i;
    int *array;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    array = (int *)malloc(n * sizeof(int));

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &array[i]);

    heapSort(array, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", array[i]);

    printf("\n");

    free(array);
    return 0;
}
```

Executed Output

```
Terminal — 17_heap_sort.c

$ gcc 17_heap_sort.c -o program && ./program
Enter number of elements: Enter elements: Sorted array: 10 20 30 40 50 60
70 80
```